

glmnet (version 4.1-8)

cv.glmnet: Cross-validation for glmnet

Description

Does k-fold cross-validation for glmnet, produces a plot, and returns a value for ``lambda`` (and ``gamma`` if ``relax=TRUE``)

Usage

```
cv.glmnet(  
  x,  
  y,  
  weights = NULL,  
  offset = NULL,  
  lambda = NULL,  
  type.measure = c("default", "mse", "deviance", "class", "auc", "mae", "C"),  
  nfolds = 10,  
  foldid = NULL,  
  alignment = c("lambda", "fraction"),  
  grouped = TRUE,  
  keep = FALSE,  
  parallel = FALSE,  
  gamma = c(0, 0.25, 0.5, 0.75, 1),  
  relax = FALSE,  
  trace.it = 0,  
  ...  
)
```

Value

an object of class ``"cv.glmnet"`` is returned, which is a list with the ingredients of the cross-validation fit. If the object was created with ``relax=TRUE`` then this class has a prefix class of ``"cv.relaxed"``.

<code>lambda</code>	the values of <code>`lambda`</code> used in the fits.
<code>cvm</code>	The mean cross-validated error - a vector of length <code>`length(lambda)`</code> .
<code>cvsd</code>	estimate of standard error of <code>`cvm`</code> .
<code>cvup</code>	upper curve = <code>`cvm+cvsd`</code> .
<code>cvlo</code>	lower curve = <code>`cvm-cvsd`</code> .
<code>nzero</code>	number of non-zero coefficients at each <code>`lambda`</code> .
<code>name</code>	a text string indicating type of measure (for plotting purposes).
<code>glmnet.fit</code>	a fitted glmnet object for the full data.
<code>lambda.min</code>	value of <code>`lambda`</code> that gives minimum <code>`cvm`</code> .
<code>lambda.1se</code>	largest value of <code>`lambda`</code> such that error is within 1 standard error of the minimum.
<code>fit.preval</code>	

foldid if ``keep=TRUE``, this is the array of prevalidated fits. Some entries can be ``NA``, if that and subsequent values of ``lambda`` are not reached for that fold

if ``keep=TRUE``, the fold assignments used

index a one column matrix with the indices of ``lambda.min`` and ``lambda.1se`` in the sequence of coefficients, fits etc.

relaxed if ``relax=TRUE``, this additional item has the CV info for each of the mixed fits. In particular it also selects ``lambda``, ``gamma`` pairs corresponding to the 1se rule, as well as the minimum error. It also has a component ``index``, a two-column matrix which contains the ``lambda`` and ``gamma`` indices corresponding to the "min" and "1se" solutions.

Arguments

x ``x`` matrix as in ``glmnet``.

y response ``y`` as in ``glmnet``.

weights Observation weights; defaults to 1 per observation

offset Offset vector (matrix) as in ``glmnet``

lambda Optional user-supplied lambda sequence; default is ``NULL``, and ``glmnet`` chooses its own sequence. Note that this is done for the full model (master sequence), and separately for each fold. The fits are then aligned using the master sequence (see the ``alignment`` argument for additional details). Adapting ``lambda`` for each fold leads to better convergence. When ``lambda`` is supplied, the same sequence is used everywhere, but in some GLMs can lead to convergence issues.

type.measure loss to use for cross-validation. Currently five options, not all available for all models. The default is ``type.measure="deviance"``, which uses squared-error for gaussian models (a.k.a ``type.measure="mse"`` there), deviance for logistic and poisson regression, and partial-likelihood for the Cox model. ``type.measure="class"`` applies to binomial and multinomial logistic regression only, and gives misclassification error. ``type.measure="auc"`` is for two-class logistic regression only, and gives area under the ROC curve. ``type.measure="mse"`` or ``type.measure="mae"`` (mean absolute error) can be used by all models except the ``"cox"``; they measure the deviation from the fitted mean to the response. ``type.measure="C"`` is Harrel's concordance measure, only available for ``cox`` models.

nfolds number of folds - default is 10. Although ``nfolds`` can be as large as the sample size (leave-one-out CV), it is not recommended for large datasets. Smallest value allowable is ``nfolds=3``

foldid an optional vector of values between 1 and ``nfolds`` identifying what fold each observation is in. If supplied, ``nfolds`` can be missing.

alignment This is an experimental argument, designed to fix the problems users were having with CV, with possible values ``"lambda"`` (the default) else ``"fraction"``. With ``"lambda"`` the ``lambda`` values from the master fit (on all the data) are used to line up the predictions from each of the folds. In some cases this can give strange values, since the effective ``lambda`` values in each fold could be quite different. With ``"fraction"`` we line up the predictions in each fold according to the fraction of progress along the regularization. If in the call a ``lambda`` argument is also provided, ``alignment="fraction"`` is ignored (with a warning).

grouped	This is an experimental argument, with default <code>TRUE</code> , and can be ignored by most users. For all models except the <code>"cox"</code> , this refers to computing <code>nfolds</code> separate statistics, and then using their mean and estimated standard error to describe the CV curve. If <code>grouped=FALSE</code> , an error matrix is built up at the observation level from the predictions from the <code>nfolds</code> fits, and then summarized (does not apply to <code>type.measure="auc"</code>). For the <code>"cox"</code> family, <code>grouped=TRUE</code> obtains the CV partial likelihood for the Kth fold by <i>subtraction</i> ; by subtracting the log partial likelihood evaluated on the full dataset from that evaluated on the on the (K-1)/K dataset. This makes more efficient use of risk sets. With <code>grouped=FALSE</code> the log partial likelihood is computed only on the Kth fold
keep	If <code>keep=TRUE</code> , a <i>prevalidated</i> array is returned containing fitted values for each observation and each value of <code>lambda</code> . This means these fits are computed with this observation and the rest of its fold omitted. The <code>foldid</code> vector is also returned. Default is keep=FALSE. If <code>relax=TRUE</code> , then a list of such arrays is returned, one for each value of 'gamma'. Note: if the value 'gamma=1' is omitted, this case is included in the list since it corresponds to the original 'glmnet' fit.
parallel	If <code>TRUE</code> , use parallel <code>foreach</code> to fit each fold. Must register parallel before hand, such as <code>doMC</code> or others. See the example below.
gamma	The values of the parameter for mixing the relaxed fit with the regularized fit, between 0 and 1; default is <code>gamma = c(0, 0.25, 0.5, 0.75, 1)</code>
relax	If <code>TRUE</code> , then CV is done with respect to the mixing parameter <code>gamma</code> as well as <code>lambda</code> . Default is <code>relax=FALSE</code>
trace.it	If <code>trace.it=1</code> , then progress bars are displayed; useful for big models that take a long time to fit. Limited tracing if <code>parallel=TRUE</code>
...	Other arguments that can be passed to <code>glmnet</code>

Author

Jerome Friedman, Trevor Hastie and Rob Tibshirani
Noah Simon helped develop the 'coxnet' function.
Jeffrey Wong and B. Narasimhan helped with the parallel option
Maintainer: Trevor Hastie hastie@stanford.edu

Details

The function runs `glmnet` `nfolds`+1 times; the first to get the `lambda` sequence, and then the remainder to compute the fit with each of the folds omitted. The error is accumulated, and the average error and standard deviation over the folds is computed. Note that `cv.glmnet` does NOT search for values for `alpha`. A specific value should be supplied, else `alpha=1` is assumed by default. If users would like to cross-validate `alpha` as well, they should call `cv.glmnet` with a pre-computed vector `foldid`, and then use this same fold vector in separate calls to `cv.glmnet` with different values of `alpha`. Note also that the results of `cv.glmnet` are random, since the folds are selected at random. Users can reduce this randomness by running `cv.glmnet` many times, and averaging the error curves.

If `relax=TRUE` then the values of `gamma` are used to mix the fits. If η is the fit for lasso/elastic net, and η_R is the relaxed fit (with unpenalized coefficients), then a relaxed fit mixed by γ is

$$\eta(\gamma) = (1 - \gamma)\eta_R + \gamma\eta.$$

There is practically no extra cost for having a lot of values for `gamma`. However, 5 seems sufficient for most purposes. CV then selects both `gamma` and `lambda`.

References

Friedman, J., Hastie, T. and Tibshirani, R. (2008) *Regularization Paths for Generalized Linear Models via Coordinate Descent* (2010), *Journal of Statistical Software*, Vol. 33(1), 1-22, tools:::Rd_expr_doi("10.18637/jss.v033.i01").
Simon, N., Friedman, J., Hastie, T. and Tibshirani, R. (2011) *Regularization Paths for Cox's Proportional Hazards Model via Coordinate Descent*, *Journal of Statistical Software*, Vol. 39(5), 1-13, tools:::Rd_expr_doi("10.18637/jss.v039.i05").

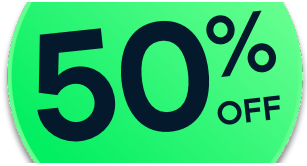
See Also

``glmnet`` and ``plot``, ``predict``, and ``coef`` methods for ``"cv.glmnet"`` and ``"cv.relaxed"`` objects.

Examples

```
set.seed(1010)
n = 1000
p = 100
nzc = trunc(p/10)
x = matrix(rnorm(n * p), n, p)
beta = rnorm(nzc)
fx = x[, seq(nzc)] %*% beta
eps = rnorm(n) * 5
y = drop(fx + eps)
px = exp(fx)
px = px/(1 + px)
ly = rbinom(n = length(px), prob = px, size = 1)
set.seed(1011)
cvob1 = cv.glmnet(x, y)
plot(cvob1)
coef(cvob1)
predict(cvob1, newx = x[1:5, ], s = "lambda.min")
```

Run this code



Buy Now >



```
#####
title("Gaussian Family", line = 2.5)
set.seed(1011)
par(mfrow = c(2, 2), mar = c(4.5, 4.5, 4, 1))
cvob2 = cv.glmnet(x, ly, family = "binomial")
plot(cvob2)
title("Binomial Family", line = 2.5)
frame()
set.seed(1011)
cvob3 = cv.glmnet(x, ly, family = "binomial", type.measure = "class")
plot(cvob3)
title("Binomial Family", line = 2.5)
if (FALSE) {
cvob1r = cv.glmnet(x, y, relax = TRUE)
plot(cvob1r)
predict(cvob1r, newx = x[, 1:5])
set.seed(1011)
cvob3a = cv.glmnet(x, ly, family = "binomial", type.measure = "auc")
plot(cvob3a)
title("Binomial Family", line = 2.5)
set.seed(1011)
mu = exp(fx/10)
y = rpois(n, mu)
cvob4 = cv.glmnet(x, y, family = "poisson")
plot(cvob4)
title("Poisson Family", line = 2.5)

# Multinomial
n = 500
p = 30
nzc = trunc(p/10)
x = matrix(rnorm(n * p), n, p)
beta3 = matrix(rnorm(30), 10, 3)
beta3 = rbind(beta3, matrix(0, p - 10, 3))
f3 = x %*% beta3
p3 = exp(f3)
p3 = p3/apply(p3, 1, sum)
g3 = glmnet::rmult(p3)
set.seed(10101)
cvfit = cv.glmnet(x, g3, family = "multinomial")
plot(cvfit)
title("Multinomial Family", line = 2.5)
```

```
# Cox
beta = rnorm(nzc)
fx = x[, seq(nzc)] %*% beta/3
hx = exp(fx)
ty = rexp(n, hx)
tcens = rbinom(n = n, prob = 0.3, size = 1) # censoring indicator
y = cbind(time = ty, status = 1 - tcens) # y=Surv(ty,1-tcens) with library(survival)
foldid = sample(rep(seq(10), length = n))
fit1_cv = cv.glmnet(x, y, family = "cox", foldid = foldid)
plot(fit1_cv)
title("Cox Family", line = 2.5)

# Parallel
require(doMC)
registerDoMC(cores = 4)
x = matrix(rnorm(1e+05 * 100), 1e+05, 100)
y = rnorm(1e+05)
system.time(cv.glmnet(x, y))
system.time(cv.glmnet(x, y, parallel = TRUE))
}
```

Run the code above in your browser using [DataCamp Workspace](#)

Powered by [DataCamp](#)